return an object with several name/value pairs. In this case, the object that's initialized as `data` has a name/value pair of `tweets: [Array of Data]`. That's why we need to refer to `data.tweets` after we've loaded tweets.json, but refer to `data` when we load cities.csv. The structure of tweets.json highlights this distinction.

Both d3.csv and d3.json are asynchronous, and will return after the request to open the file and not after processing the file. Loading a file, which is typically an operation that takes more time than most other functions, won't be complete by the time other functions are called. If you call functions that require the loaded data before it's loaded, then they'll fail. You can get around this asynchronous behavior in two ways. You can nest the functions operating on the data in the data-loading function:

```
d3.csv("somefiles.csv", function(data) {doSomethingWithData(data)});
```

Or you can use a helper library like queue.js (which we'll use in chapter 7) to trigger events upon completion of the loading of one or more files. You'll see queue.js in action in later chapters. Note that `d3.csv()` has a method `.parse()` that you can use on a block of text rather than an external file. If you need more direct control over getting data, you should review the documentation for `d3.xhr()`, which allows for more fine-grained control of sending and receiving data.

2.1.2 Formatting data

After you load the datasets, you'll need to define methods so that the attributes of the data directly relate to settings for color, size, and position graphical elements. If you want to display the cities in the CSV, you probably want to use circles, size those circles based on population, and then place them according to their geographic coordinates. We have long-established conventions for representing cities on maps graphically, but the same can't be said about tweets. What graphical symbol to use to represent a single tweet, how to size it, and where to place it are all open questions. To answer these questions, you need to understand the forms of data you'll encounter when doing data visualization. Programming languages and ontologies define numerous datatypes, but it's useful to think of them as quantitative, categorical, geometric, temporal, topological, or raw.

QUANTITATIVE

Numerical or quantitative data is the most common type in data visualization. Quantitative data can be effectively represented with size, position, or color. You'll typically need to normalize quantitative data (the second step in creating data visualization shown in figure 2.4) by defining scales using `d3.scale()`, as explained in section 2.1.3,

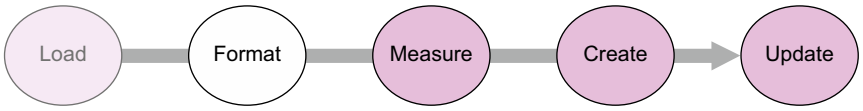


Figure 2.4 After loading data, you need to make sure that it's formatted in such a way that it can be used by various JavaScript functions to create graphics.